

Customizing the coreBuilder and coreConsultant Activity Flow

coreBuilder Application Note

Mark Noll, Synopsys Design Reuse Group

July 2000

Abstract

The coreBuilder tools must be able to support a wide variety of core development and core integration flows. In order make this possible, the tools can be customized in a number of different ways. This application note discusses how to customize the tool by adding new activities to a flow or by modifying existing activities. It also discusses plug-in knowledge databases, which provide a mechanism for associating customizations with a particular tool, core, or customer.

Introduction

The coreBuilder and coreConsultant tools define flows for completing core development/packaging and core integration, respectively. In order to support the wide variety of development and integration flows in use in the industry today, the flows within the tools can be customized so that existing steps (activities) can be modified to perform extra work and/or new steps (activities) can be added. This application note will describe how to define and then utilize customizations to the flows in both tools.

Customizations can be made to be applicable for an entire site, a specific product (coreBuilder or coreConsultant), a specific core, or even a specific user. At a high level, the process of customizing a flow involves writing TCL procedures to implement the customization, and (optionally) to provide a user interface to the custom functionality.

The remainder of this document is broken down into the following four parts.

- Overview of activities
- Overview of plug-in knowledge databases
- How to modify an existing activity
- How to add a custom activity

Overview of Activities

Each of the entries in the coreBuilder (or coreConsultant) activity list is a unique activity. Activities have a number of interesting properties.

- name – used to reference the activity in a batch TCL script. For example, TechSetup is an activity in both coreBuilder and coreConsultant.

- label – used in GUI to represent the activity. For example, the TechSetup activity has the label “Reference Technology” in coreBuilder, and “Technology Setup” in coreConsultant.
- state – either complete or incomplete. This maps to whether to associated check-box in the activity dialog is checked off or not.
- prerequisites – activities which must be completed before this activity can be enabled. As an example, the coreConsultant activity SpecifyConfiguration has the TechSetup activity as a pre-requisite. Therefore it is not enabled in the GUI until TechSetup has been completed.
- parameters – objects used to capture user input for the activity. For example, the TechSetup activity contains the parameters: SearchPath, TargetLibrary, and LinkLibrary, among others. The user interface to an activity is defined by its parameters and their associated types (string, integer, list, etc.).

When an activity is selected in the GUI, the following process is followed:

```
execute pre-activity command (if defined)
show dialog containing activity parameters
if { user selected Okay button } {
    validate parameter values
    execute post-activity command (if defined)
} else {
    execute cancel-activity command (if defined)
}
```

If any of the commands causes a TCL error, processing for the activity stops. If the parameter value checking shows an error, the parameter dialog will remain posted and focus within the window will shift to the first parameter containing an error.

Aside from the commands used to create and modify activities, which are defined below, there are two useful commands for listing activities and their associated

parameters: `report_activities`, and `report_activity_parameters`, respectively. See the man pages for these commands for usage details.

Overview of Plug-in Knowledge Databases

A plug-in knowledge database is simply a knowledge database stored in one of a set of pre-defined locations such that it gets loaded into the tool at start up time or core loading time. As in tools like Netscape, a plug-in KB provides extra functionality to the tool it is loaded into.

Plug-in knowledge databases are loaded automatically when found in directories defined by a set of environment variables.

`RT_PLUGINS` – defines a directory containing mode independent plug-ins (apply to both `coreBuilder` and `coreConsultant`). The default for this variable is `<tool installation root>/libraries/dware/plugins`.

`RT_BUILDER_PLUGINS` – defines a directory containing plug-ins for `coreBuilder`. The default for this variable is `<tool installation root>/libraries/dware/plugins/builder`.

`RT_CONSULTANT_PLUGINS` – defines a directory containing plug-ins for `coreConsultant`. The default for this variable is `<tool installation root>/libraries/dware/plugins/consultant`

In addition to the mode independent, and mode dependent plug-ins indicated above, plug-ins can also be core specific. Core specific plug-ins are specified as part of the `coreBuilder` activity flow (the `LoadConsultantPlugins` activity) and are automatically loaded into `coreConsultant` when the resulting core is loaded.

Plug-ins are created by writing the TCL code to be contained in the plug-in (one or more files in a single directory) and then packaging the code as a plug-in KB. TCL code for a plug-in can be divided into two basic functions:

- Code that defines objects that will reside in the plug-in KB. The primary example of this would be custom activities.
- Code that defines procedures or sets global variables which are to be available when the plug-in is actually being used.

The primary difference between the two types of code is when the code gets evaluated. Code for defining or modifying objects needs to be evaluated as part of building the plug-in KB. Procedures and variables that need to be set when the tool is actually being used (for core development or integration) needs to be evaluated when the plug-in KB is loaded as part of a core development or core integration session. The code is separated into these two groups based on file name. All of the TCL code for a plug-in KB must be in files ending with “.tcl”. Files that end with “Obj.tcl” are assumed to contain TCL that is to be evaluated while the plug-in is being built but not when the plug-in is being loaded for use. All other files ending with “.tcl” are evaluated while the plug-in is being built and while the plug-in is being loaded for use.

Once all of the TCL code has been written, the plug-in KB is built by doing the following:

- Make sure that all files in the plug-in directory ending with “.tcl” are needed for the plug-in.
- Build the plug-in with the following command:

```
coreConsultant -x "create_plugin_kb -name <kb name>
[<TCL directory>]; quit"
```

If no directory is specified, the current directory is assumed. The plug-in KB will be written out to the file “<kb name>.kb” in the plug-in directory. All that is required to use the plug-in is to insert it into the proper plug-in directory as defined at the beginning of this section. Note that this step is not needed for core specific plug-ins since the build of the plug-in is handled internally for that case.

How to Modify an Existing Activity

Existing activities can be modified using the command `add_activity_hook`. This command can be used to add to an existing pre-activity, post-activity, or cancel-activity command. These commands are just TCL command strings, and `add_activity_hook` allows you to add to either the end (by default) or the beginning of the existing command string. For example, consider the following:

```
add_activity_hook -before -prepend TechSetup {echo "Hello"}
add_activity_hook -after          TechSetup {echo "Okay"}
add_activity_hook -cancel         TechSetup {echo "Cancel"}
```

As part of a plug-in, this code needs to be put in a file ending with `Obj.tcl` since it modifies an object (activities). Assume the code is put in the file `testObj.tcl` and then built as a plug-in and loaded into `coreBuilder`. Then when the `TechSetup` activity is selected, the string "Hello" will be printed before the normal `TechSetup` pre-activity command is executed. If the parameter dialog associated with `TechSetup` is closed via the Cancel button, the message "Cancel" will be printed after the normal `TechSetup` cancel-activity command is executed. Similarly, the message "Okay" will be printed after the normal `TechSetup` post-activity command is executed if the parameter dialog is closed using the Okay button. See the `add_activity_hook` man page for details about the command line options.

How to Add a Custom Activity

Custom activities can be defined by creating a file of TCL commands that define an activity, and a set of (optional) associated parameters. Also included in this file would be specification of a number of attributes on both the activity and the parameters. All of these commands would need to be placed in a file ending with "`Obj.tcl`" since they are defining objects. In addition, one or more other TCL files would be

defined to contain the action procedures (pre-activity, post-activity, and cancel-activity) as needed. The best way to describe this is by example. The TCL code below defines a custom activity.

```
# Define activity for setting up RTL scan insertion.
set A [create_custom_activity \
    -name ScanInsertion \
    -type Global \
    -label "Add scan chain to core." \
    -pre "verifyReadyForChain" \
    -post "addScanChain" \
    -description "Activity for adding scan chains" \
    -url [file join docs scan help.html]]

# Indicates the number of scan chains to be inserted.
set P [create_custom_activity_parameter \
    -activity $A \
    -name NumberOfChains \
    -type integer \
    -label "Number of Scan Chains" \
    -description \
        {Specify the desired number of scan chains to
be inserted.} \
    -default 2]

# Indicates the insertion mode to be used.
set P [create_custom_activity_parameter \
    -activity $A \
    -name InsertionMode \
    -type string \
    -label "Insertion Mode" \
    -description {Defines the scan chain insertion
mode.}]
set_attribute $P -attr EnumValues \
    -value "Basic Advanced"
set_attribute $P -attr DefaultValue -value "Basic"

# Set up requirements and sequencing. This activity
# can't be completed until after configuration, and
# must be completed before generating
# synthesis scripts.
complete_custom_activity_definition -activity $A \
    -sequence 200 \
    -prereqs SpecifyConfiguration \
    -postreqs Synthesize
```

The three commands used above are all that are required (aside from `set_attribute`) to build up custom activities for either the developer or integrator flow.

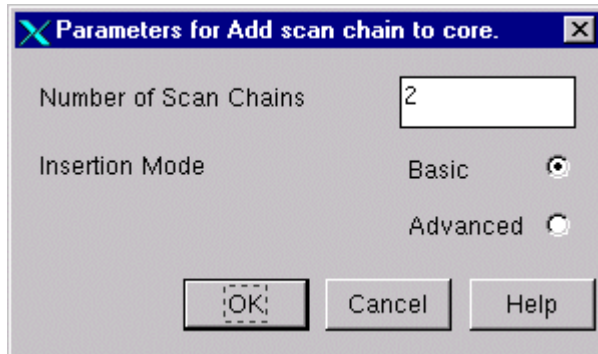
The first command (`create_custom_activity`) defines a new activity, specifies the label to be shown in the activity checklist, defines short help (available via right-mouse-button) and detailed help (available via Help button), and specifies the pre-activity and post-activity command strings. The procedures `verifyReadyForScanChain` and `addScanChain` should be defined in another TCL file in the same directory so they get included in the plug-in.

The next two commands (calls to `create_custom_activity_parameter`) define parameters needed to gather information from the user. The first is used to indicate how many scan chains should be inserted and the second is used to define the insertion mode. Additional attributes can be set on the parameters to control the look of the associated parameter dialog. See the application note 'Parameters and the Automatic Dialog Builder' for details.

The final command (`complete_custom_activity_definition`) defines how the activity fits it with the other activities. In this case, the custom activity will not be enabled until `SpecifyConfiguration` has been completed, and `Synthesize` will not be enabled until after `ScanInsertion` has been completed. In addition, a sequence number is specified to control where in the GUI the activity shows up. Activities are shown in order sorted by sequence. You can use the command "`report_attribute [find_item -type activity] -attr Sequence`" to list the sequence number of existing activities. See the man pages for these commands for more detail about the specific command line options.

When a user selects this activity in `coreConsultant`, `verifyReadyForScanChain` will be called. If that command returns without error, then a dialog will be posted (as in the figure below) containing a value field for entry of

the number of scan chains and a drop-down box for selecting the scan insertion mode.



Assuming the user closes the dialog using the Okay button, the addScanChain command will then be called to actually insert the scan chain. The pre- and post-activity commands for this activity might look something like the following.

```
proc verifyReadForChain {} {
    # Make sure RTL insertion tool is available
    set result [exec which insert_scan_tool]
    if { ![file exists $result] } {
        return -code error \
            "Can't locate: insert_scan_tool."
    }
}

proc addScanChain {} {
    # Get parameter values.
    set chains [get_activity_parameter ScanInsertion \
        NumberOfChains]
    set mode [get_activity_parameter ScanInsertion \
        InsertionMode]

    # Get source code to be operated upon and then pass
    # it to the RTL scan chain insertion tool.
    set files [get_hdl_file_list]
    set rc [catch {exec insert_scan_tool -mode $mode \
        -chains $chains -files $files}]

    if {$rc} {
        return -code error "insert_scan_tool failed."
    }
}
```

These procedures utilize a number of useful utility commands

available for plug-in development. Please refer to the API man page for details about commands available for use during plug-in development.